Super-Precise Latency: How CPU Threading Distorts Benchmarks, an Industry-Wide Audit, and How to Fix It

Gustavo Pedro Ricou and Romaric Duvignau

Abstract—Engineers choose message brokers, which pass data between programs, partly by benchmarks reporting latencies below a millisecond. At that scale, when a program reads the clock matters. A broker acknowledges each message to the producer and delivers it to the consumer, two readings on one machine's clock. Read as a latency, their difference understates end-to-end latencies several-fold and distorts broker comparisons. Measured, it spreads over milliseconds; part falls below zero. On one broker the acknowledgment arrives first, and there the cause is scheduling: the thread reading it can wait for a processor core longer than the message takes to reach the consumer. The other broker often delivers first. The rate of values below zero obeys laws we derive and test by manipulation: it rises with processor load, whose placement matters, falls when clock-reading threads get real-time priority, and falls on longer paths, on the timescale of the scheduler's slice. An industry-wide audit finds such values discarded uncounted or never measured, and a millisecond-clock filter deleting the fastest messages. Kernel timestamps, read on a tool's own Linux sockets, exclude those threads' wait at modest rates. We give benchmark authors and readers practical checks and measure what most buy.

Index Terms—Apache Kafka; benchmark auditing; kernel timestamping; latency benchmarking; message brokers; Redis Streams; thread scheduling; timestamp resolution.

1 Introduction

Many online services move their data through a message broker, a service that takes each message from the program that publishes it, the *producer*, and delivers it to the program that receives it, the *consumer* (Fig. 1a). Engineers choose among brokers partly by latency benchmarks [1], which time each message from producer to consumer and now report latencies below a millisecond [2]. At that scale, a thread's wait for a processor core can exceed the latency itself. A team that ranks brokers or promises a response time on such a number may be acting on how the number was measured, not on the broker. This paper shows how to separate the two and measure the broker alone.

The stakes reach beyond any one benchmark. Delays of microseconds now shape how datacenter systems are built [3], and a large service is only as fast as the slowest of its many parts [4], so its designers need latencies they can trust. Yet experiments on computer systems can produce wrong data with nothing obviously wrong [5], a concern across the sciences [6]. Timed from the broker's acknowledgment, as our first benchmark did, a message's latency understates its trip from producer to consumer several-fold and distorts broker comparisons (Section 6).

We study one difference. Once the broker has written a message, it returns an acknowledgment to the producer and delivers the message to the consumer. Let the producer and the consumer run on one machine and read its one clock. The producer reads the acknowledgment at t_{ack}, the consumer reads the message at t_{recv}, and on a broker that acknowledges first the two should come in that order, apart only by a small positive margin, the time the message still needs to reach the consumer. We make four contributions.

1) **The distribution** (Section 3). On one clock, $S = t_{\mathrm{recv}} - t_{\mathrm{ack}}$ is spread over milliseconds, and 8.79% of it falls below zero. Nothing arrived early: no latency timed from the *publish call*, the moment the producer hands the message over, falls below zero.
2) **The mechanism, and its laws** (Section 4). A thread must wait for a core before it can read the clock. On Kafka, whose acknowledgment the kernel timed arriving first, the thread that reads it can wait longer than the message takes to reach the consumer, while Redis often delivers first and needs no wait. The rate of values of S below zero rises with processor load and depends on the load's placement, falls on longer paths, and has the timescale of the scheduler's slice, set by the CPU count. *Real-time priority* runs the threads that read the clock ahead of every thread at the default *normal priority*. In every matched pair, real-time priority cut that rate at least 4.4-fold on Kafka and 4.2-fold on Redis, at unchanged *background load*, the processor work run beside the clients.
3) **The industry-wide audit** (Section 5). Of the 24 message tools we read at source, 19 cut such values out of a figure they report, and all but one, the Storage Benchmark Kit (SBK), count none. Under the strictest reading of our classification, twelve still cut them. Of the 32 tools we ran, 19 take both timestamps in one process, where no clock offset can reach them. But eight of those time a round trip and

- *G. P. Ricou is with the School of Computer Science and Statistics, Trinity College Dublin, Dublin 2, Ireland. E-mail: pedrorig@tcd.ie.*
- *R. Duvignau is with the Department of Computer Science and Engineering, Chalmers University of Technology, Gothenburg, Sweden. E-mail: duvignau@chalmers.se.*

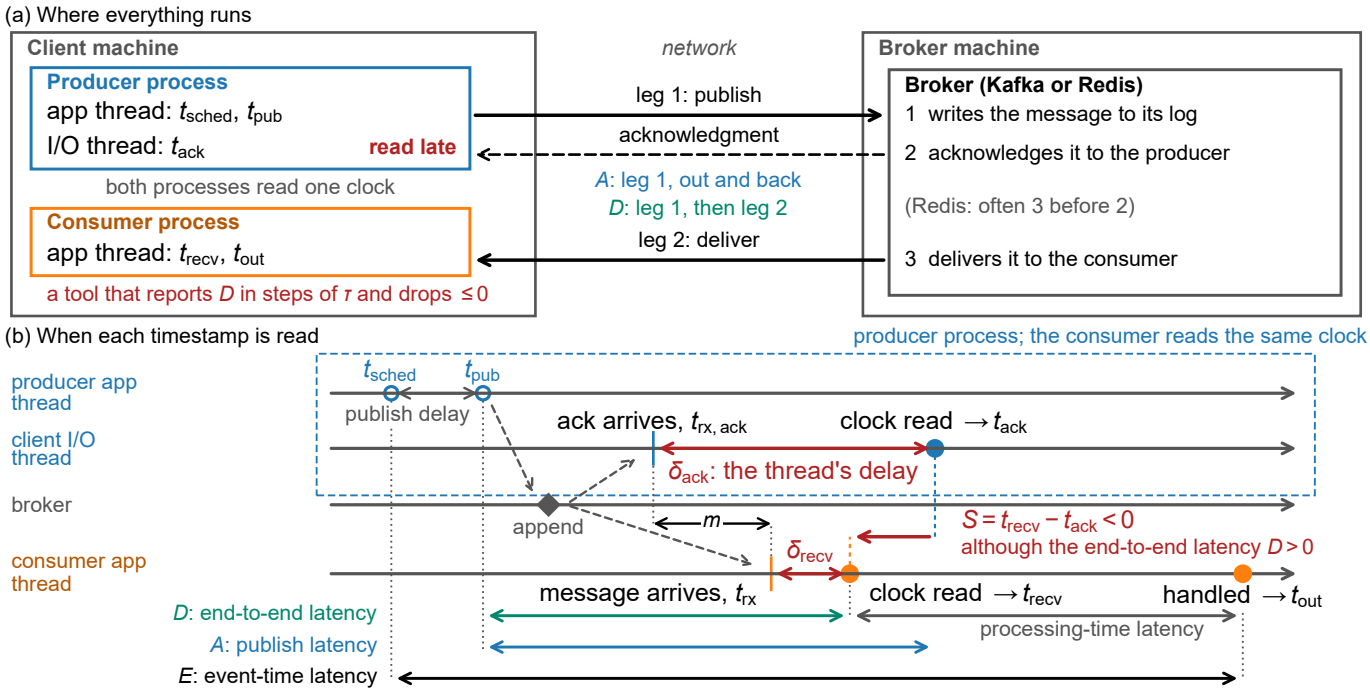


Fig. 1. **Every message crosses two legs, and a thread may wait for a core before it reads the clock.** (a) The producer and consumer processes share a client machine and its one clock, and the broker runs on a second machine. Leg 1 carries a message to the broker, which writes it to its log and returns an acknowledgment. Leg 2 carries it on to the consumer. (b) The producer reads t_{pub} as it publishes. The acknowledgment and the message reach the client machine at the kernel's receive timestamps $t_{\text{rx,ack}}$ and t_{rx}, the margin m apart (Section 4.1). The client's I/O thread reads t_{ack} only after its delay δ_{ack}, which holds any wait for a core, and the consumer reads t_{recv} after its own delay δ_{recv}. Here A runs past D, so $S = D - A$ is below zero although nothing arrived early. The red notes mark this late reading and, below the consumer's box, a tool whose clock advances in steps of τ and that drops every value at or below zero (Section 5). E runs from t_{sched} to t_{out} (Section 2). The path drawn is Kafka's. On Redis the application thread reads the acknowledgment, and in most runs the delivery arrives first, so $m < 0$.

four only the acknowledgment, neither of which is a delivery's *one-way latency*, its time from sender to receiver [7], and the rest cannot see a separate receiving process's wait. On a millisecond clock, a filter that keeps only latencies above zero, a *positivity filter*, also deletes the fastest messages, and no public report we audited says how many samples it kept.

4) **What to do** (Section 6). Checks for benchmark authors and readers, each with what it buys where we measured it, and one fix for a Linux tool. The fix times each message between the kernel's timestamps as it leaves the client machine and arrives back, which a tool can read on its own sockets in four steps. That latency held still across background load and priority at fifty messages a second on one pair of machines, its median within the 10% set in advance, and agreed with packet captures up to 2,000 a second.

Together these show that a broker latency below a millisecond describes both the broker and how it was timed, and they give the means to time the broker alone.

2 How a Message Is Timed

A producer process publishes messages to a broker at a paced rate, and a consumer process reads them back. Each message crosses two legs (Fig. 1a). Leg 1 carries it to the broker, which writes it to its log and returns an acknowledgment along leg 1, and leg 2 carries it on to the consumer. Three timestamps bracket its path: t_{pub}, read immediately before the publish call; t_{ack}, read when the producer learns that the broker has accepted the message; and t_{recv}, read when the consumer holds it. We call any difference of two timestamps a *span*. Three spans follow:

$$D \equiv t_{\text{recv}} - t_{\text{pub}}, \qquad A \equiv t_{\text{ack}} - t_{\text{pub}},$$
$$S \equiv t_{\text{recv}} - t_{\text{ack}} \;=\; D - A. \tag{1}$$

Broker benchmarks call D the *end-to-end latency*, the message's time from the publish call to the consumer, and A the *publish latency*, the producer's wait to hear that the broker has it [1]. S compares the acknowledgment's return along leg 1 with the message's trip along leg 2. On Kafka the broker returns the acknowledgment before it answers the consumer's waiting request. Redis writes the delivery to a waiting consumer first. Some brokers can deliver a message before confirming it (Supplement S3.6). A benchmark that starts its clock at the acknowledgment, as our own first one did, reports S as the message's latency, and every publish latency carries whatever delays the reading of its acknowledgment (Section 5.2). Two more readings place a message in its workload: t_{sched}, when the replayed schedule made it due, and t_{out}, when the consumer had handled it. The *event-time latency* $E = t_{\text{out}} - t_{\text{sched}}$ holds D, with the *publish delay* $t_{\text{pub}} - t_{\text{sched}}$ before it and the consumer's *processing-time latency* $t_{\text{out}} - t_{\text{recv}}$ after it [8].

Unless the text says otherwise, the measurements come from Oracle Cloud virtual machines on a private network. One runs both brokers: Kafka 4.1.1 as a single node with one

partition and one replica, so that `acks=all` is the leader's own append, and Redis 7 Streams, read through a consumer group. Another, the *client machine*, runs the producer and the consumer as separate CPython processes through the kafka-python and redis-py client libraries (versions, where recorded, in Supplement S1), and both read one clock, so nothing here depends on synchronizing clocks. In a control the brokers ran on the client machine itself (Supplement S1). Its kernel is Linux 6.8, whose scheduler is EEVDF (Earliest Eligible Virtual Deadline First) [9].

The producer replays StatsBomb's open football event data [10]; only its message sizes and rates matter here. Each message is a JSON record of an event's identifiers and timestamps, 253–321 bytes, which one experiment pads by up to 256 KB. In nine runs out of ten a producer publishes 0.60–1.03 messages a second, as a sparse event feed does, and up to 12 producers run at once. *Utilization* ρ is the fraction of processor time busy on the client machine over a run, set by the background load, which runs only there.

A *condition* is one point of the experimental design: a broker, a publish rate, a message size, a background load and how often the consumer confirms what it has read. A *run* executes a condition once, a *replicate* is a further run of it, and a *campaign* is a set of runs designed together. Where an experiment changes one variable and holds the rest, each value it gives that variable is a *configuration*. A later campaign ran on three pairs of Microsoft Azure machines, two x86 and one Neoverse N2, with the same brokers and clients (`kafka-python` 2.3.2, `redis-py` 7.4.1) and Kafka's Java client, `kafka-clients` 3.9.0. It was *registered*: its predictions were committed to the project's public repository before the runs that test them [11] (Supplement S6.1). Supplement S1.4 lists the runs behind each number.

3 The Distribution of S

On a broker that acknowledges first, S is ideally a small positive margin (Section 4.1). Measured, it is spread over milliseconds, and part of it falls below zero (Fig. 2). Over the 708,505 messages of the 5,863 cloud runs that produced matched records, 62,264 values of S, 8.79%, fall below zero (excluding 30,225 messages delayed by a testbed fault; Supplement S1.5). We call this share the *negative-span rate*. It pools runs at every load, each weighted by its messages. Per run it goes from zero, in 1,314 runs, to 55%, with a median of 3.6%. It rises with utilization and falls at real-time priority (Fig. 2b). Both brokers show it, Kafka on 8.87% of its 359,766 messages and Redis on 8.71% of its 348,739, with per-run medians of 2.1% and 5.9%. The registered campaign found it again in Kafka's own Java client, at a third to a quarter of Python's rate, and on Neoverse N2 as on x86 (Supplement S6.2).

Nothing arrived early. On the same messages and the same clock, neither D nor A falls below zero once, and the smallest of each is far from it, while S reaches -99.8 ms (Table 1). So the two readings came out of order: on Kafka because one was taken late, and on Redis mostly because the delivery came first (Section 4.1). When a widely run broker benchmark reported an average publish latency above its average end-to-end latency, with producer and consumer on one node, its user was told that the clocks were skewed between nodes [12]. A Yahoo Streaming Benchmark user who saw values below zero on one machine got the same answer (Supplement S5). One clock has no skew to blame.

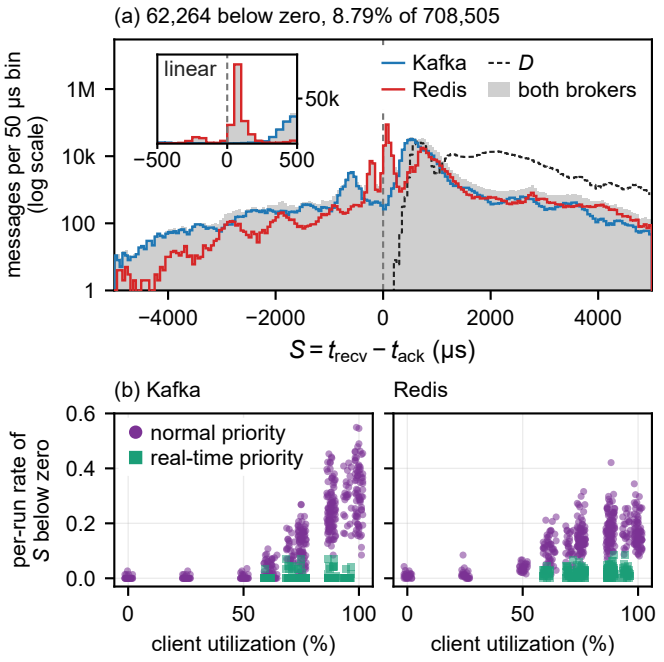


Fig. 2. S **spreads over milliseconds, and part of it falls below zero.** (a) $S = t_{\text{recv}} - t_{\text{ack}}$ for the 708,505 messages of our cloud runs, 62,264 of them below zero, read on one clock at nanosecond resolution and counted in 50 μs bins over ± 5 ms, on a log scale and, inset, a linear one. 558 values lie below the window and 19,295 above it. The black dashed line is D on the same messages, never below zero. Redis's values below zero gather in a mode near $-200\,\mu$s (Supplement S3.11). (b) The negative-span rate of each of 1,250 runs of the load and priority campaigns (Table 2), at its configuration's utilization, offset sideways: it rises with utilization at normal priority and stays low at real-time priority.

TABLE 1
Only S falls below zero, and under both brokers. Values below zero by span and broker, in all 5,863 cloud runs with matched records, on one clock, with each span's origin timestamp and its smallest value.

Span	From	Kafka	Redis	All	Least, μs
Messages		359,766	348,739	708,505	
S	t_{ack}	31,899	30,365	62,264	−99,786
share below zero		8.87%	8.71%	8.79%	
D	t_{pub}	0	0	0	220
$t_{\text{out}} - t_{\text{pub}}$	t_{pub}	0	0	0	230
A	t_{pub}	0	0	0	112
E	t_{sched}	0	0	0	467

4 The Mechanism and Its Laws

A thread must wait for a core before it can read the clock. That wait is its *scheduling delay*, the time a thread that is ready to run spends queued before a core runs it, which real-time Linux measures as scheduling latency [13]. We show, by the kernel's timestamps and by *manipulation*, moving one variable with the rest held fixed, when this wait puts S below zero and when the broker's own order does. We then state the laws that govern how often, and on what timescale.

4.1 When S falls below zero

D and A are causal chains. Each starts at the publish call, which precedes the broker's write, which precedes both the acknowledgment and the delivery. S is not a chain: its two readings are two branches of one cause, the write, and neither has to come first. Since Equation 1 holds for every message, whatever the signs,

$$S < 0 \quad \text{if and only if} \quad A > D. \tag{2}$$

If neither thread waited, S would equal the *margin* m, the time the message still needs to reach the consumer once the acknowledgment has reached the producer. Each reading is late by its thread's delay, which holds any wait for a core (Fig. 1b), so S falls below zero when the acknowledging thread's delay, δ_{ack}, exceeds the margin plus the consumer's own delay, δ_{recv}. We measured the margin as the *kernel-timed order* $S_k = t_{\mathrm{rx}} - t_{\mathrm{rx,ack}}$, between the kernel's receive timestamps of the message and of its acknowledgment, which no thread's delay enters. On Kafka the acknowledgment came first, by a median of 72–92 μs, and S_k fell below zero for under 1% of messages in 21 of 24 runs. Under load, 99.8% of the values of S below zero had $S_k \geq 0$, so they are late readings. In 16 of Redis's 24 runs with sixteen publish workers the delivery came first, by 16–51 μs at the median. Redis's margin was then below zero, and a value of S below zero needed no wait. For 95–100% of its values below zero the delivery had come first (Supplement S3.11).

4.2 How often, and what moves the rate

Each thread that reads the clock is either running, in which case its reading is late by a narrow jitter, or waiting for a core. On Kafka, from an idle host to the heaviest load before its processors are fully busy, the negative-span rate grows 61× but the jitter's width only 5×, so load mostly changes how often the thread is waiting. Write p for that probability, the thread's *waiting probability*, and G for the chance that the rest of its wait, the *residual wait*, outlasts a given time. To leading order, if the consumer's own wait is short,

$$\Pr\big[\, S < 0 \mid m \,\big] \;\approx\; p\,G(m), \tag{3}$$

the waiting probability times the chance that a residual wait outlasts the margin: the textbook decomposition of a wait, here read from the order of two clock readings. Load, its placement and priority move p; the path's length moves m.

Priority. Real-time priority for the threads that read the clock should cut their waiting probability while utilization stays fixed. Every manipulation ran both brokers, 25 runs a configuration on each (Table 2). Across eight matched pairs from 60 to 95% utilization, real-time priority cut the rate 7- to 80-fold on Kafka and 6.0- to 7.8-fold on Redis, and at least 4.4- and 4.2-fold in every pair, the smallest of the pairs' run-level lower bounds. Achieved utilization matched to within 0.002 (Supplement S3.1). That refutes any account in which utilization alone sets the rate, including the queueing form we had ourselves adopted. In the registered campaign, on other machines, real-time priority removed the values below zero on Kafka in every run, and load raised the rate in all five Kafka campaigns that tested it (Supplement S6.2).

TABLE 2
Every manipulation moves the rate on both brokers, and placement moves it in opposite directions. Each factor is one configuration's negative-span rate over another's, from 25 runs and 2,408–2,985 messages a configuration, with a 95% interval that resamples whole runs. Utilization is matched in each comparison but load's. Priority gives the range over the pairs and the smallest of their lower bounds (Supplement S3.1); padding gives the highest rate over the lowest.

Manipulation, and the ratio	Kafka	Redis
Real-time priority, eight pairs normal / real-time	7–80 [≥ 4.4]	6.0–7.8 [≥ 4.2]
Placement on six cores spread / concentrated; repeat	2.07 [1.36–3.60] 2.02 [1.33–3.49]	0.65 [0.55–0.78] 0.62 [0.51–0.76]
Padding the payload highest / lowest rate; repeat	3.8 [3.1–4.8] 4.0 [3.3–5.0]	7.9 [6.2–10.4] 5.7 [4.4–7.7]
Load, idle to 88% loaded / idle	61 [33–215]	21 [14–36]

Placement. With the background load concentrated on six of the eight cores or spread over all of them, two configurations reached the same utilization, $\rho = 0.7531$, and on both brokers they gave different rates (Supplement S3.2). On Kafka the spread load gave the higher rate, 2.07-fold [bootstrap over runs, 95%: 1.36–3.60], and 2.02-fold [bootstrap over runs, 95%: 1.33–3.49] in a full replication. Spreading the load leaves no core free, so a woken thread waits more often. On Redis it was the concentrated load that gave more values below zero, so where the load is placed matters on both brokers, in opposite directions.

Path length. A wait of a given size overlaps a short path almost entirely and a long one hardly at all. Padding the payload lengthens the time to the consumer's reading, by transfer and parsing, at nearly unchanged utilization. On Kafka it raised the median of S 74-fold while utilization varied by 0.012, and the rate fell 3.8-fold; on Redis the rate fell 7.9-fold. That is the direction G predicts, and the opposite of what stress on the host would cause (Supplement S3.3).

The wait, timed message by message. The registered campaign timed each acknowledgment's own wait for a core, with a tracer that shares no probe with the application's timestamps. Of the readings below zero, 79.4–92.0% had spent most of the time from the reply's arrival to the timestamp waiting for a core. With nothing fitted, the share of acknowledgments whose own wait outlasted the run's margin put the rate at medians of 1.11–1.30 times the measured one, and the registered prediction held on four of the six pair and broker combinations (Supplement S6.2). In the kernel-timed runs, the acknowledging thread's delay under load had a 99th percentile of 3.9 ms on Kafka and 3.2 ms on Redis, 17 and 30 times its idle value, and fell back near that value at real-time priority. In each fifth of the margin, the share of values below zero lay within a factor of 1.14 on Kafka and 1.85 on Redis of the share predicted from both threads' delays (Supplement S3.11). Our earlier kernel trace of every thread's wakeups [14], pooling both brokers' threads, is consistent with the mechanism. At thresholds from 0.25 to 2 ms, around Kafka's median of S with no load, its share of longer waits stood at 0.63–1.61 times the pooled normal-priority rate, inside the factor of three fixed before it was read (Supplement S3.5).

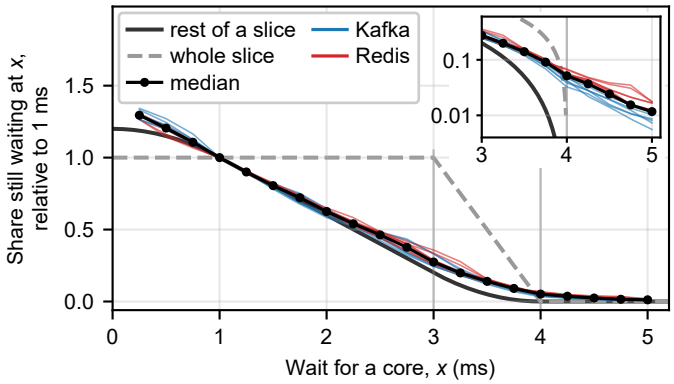


Fig. 3. **Woken by the reply, a thread waits out the rest of a slice, not a whole one.** For each of the registered campaign's twelve combinations of machine pair, broker and load (thin lines, with their median in bold), the share of acknowledgments whose thread was still waiting for a core at x, over the share at 1 ms, a tick, which puts every combination on one scale with nothing fitted. Past 1 ms waited 8.6–18.2% of acknowledgments on Kafka and 4.8–10.9% on Redis, of $1{,}516{,}342$ in 304 runs, which set the slice s by hand to 3 ms and the tick h to 1 ms (vertical lines at s and $s + h$). The dark curve waits out the rest of a slice and then up to a tick (Supplement S6.3), the dashed curve a whole slice and a tick. Inset: past 3 ms on a log scale, a tail the dark curve sets at zero. Read once the campaign's predictions had been judged.

4.3 The timescale the slice sets

A thread woken while another holds the core waits out what is left of the running thread's *slice*, the time the scheduler lets a thread run at a turn. It can wait up to a whole slice, because the scheduler does not preempt the running thread for it before that slice is spent. The kernel checks for a spent slice only at each *tick* h of its timer, 1 ms on our Oracle hosts, so the wait can run up to a tick past the slice. The kernel sets the base slice s from the CPU count n, with a cap:

$$s \;=\; s_0 \times \big(1 + \lfloor \log_2 \min(n, 8) \rfloor\big), \tag{4}$$

where s_0 is 0.75 ms in Linux 6.8 and 0.70 ms after a 2025 change [15] that Ubuntu backported to the Linux 6.8 kernels we ran (Supplement S3.7). That gives 4×0.70 ms $= 2.8$ ms on our hosts' eight CPUs. The traced waits carry that timescale: the last of their three local maxima, which survive a $2\times$ widening of the buckets, sits in the 2–4 ms bucket that contains the slice, with 10.5% of all wakeups, and above it the counts collapse (Supplement S3.7). The trace also counts waits that begin when a thread is preempted, which last about a whole slice. The thread that reads the acknowledgment is instead woken by the reply. Its own wait, timed in the registered campaign and read after the predictions were judged, is the rest of a slice (Fig. 3).

The CPU count moves the slice. In the registered campaign the slice read back at 2, 4 and 8 CPUs as Equation 4 gives with $s_0 = 0.70$ ms. The law puts the halfway point of the rate's fall inside $[s, s + h]$, from the slice to a tick past it. On Kafka the rate fell inside that band at 8 CPUs, the one count whose slice reaches its shortest end-to-end latency. On Redis a free reading, a curve required only never to rise, placed the fall there at all three counts. With the slice set by hand, that curve did so at every slice on both brokers, with slopes on s of 0.76 on Kafka and 1.01 on Redis; Section 8 gives the registered verdicts (Supplement S6.2).

A whole slice is 0.94–3.98 times the per-condition median end-to-end latencies we measured, 0.70–2.97 ms, so the slice lands on the scale of the paths now benchmarked. Together, Equations 2–4 say that the rate rises with the waiting probability and falls as the margin outgrows the residual wait, whose scale the slice sets. On our kernel any host with at least as many CPUs as ours has a slice of 2.8 ms, so a single wait can outlast any margin shorter than that.

5 An Industry-Wide Audit

Across the industry, the tools we audited delete, replace or never measure a latency at or below zero, and almost never count it (Table 3). We read 27 tools at source, ran 32 under registered predictions, measured one benchmark's filter in depth and audited the public reports made with it.

5.1 Tools read at source

We read 27 measurement tools at source, across seven languages and 20 maintainers, 14 of them organizations, and classified them with one program whose verdicts, each with its source line, are in the supplement's tool registry (Supplement S5). We chose them in rounds, not by a fixed rule: the latency tools that brokers' own projects ship and the load generators our plan named, then every other open-source latency tool we found for brokers and cloud messaging. Of them, 24 time messages or requests. Of those, 19 cut a value at or below zero out of a figure they report, in three ways: a filter drops it, the tool's own library refuses it, or another value replaces it. All but SBK count none of what they cut. Replacement hides best, because nothing looks missing. RabbitMQ PerfTest takes the absolute value [16], and `emqtt-bench` keeps every sample in its histogram but counts such a value as zero in the average it prints. Apache Kafka's own end-to-end tool hides a value of S below zero without computing one. It polls for each message only once the acknowledgment has returned, so a message that reached the consumer first is timed to that later moment, which replaces its arrival. It also rounds each latency down to whole milliseconds, so on a path faster than a millisecond every percentile it reports is zero [17]. Two of the cuts are edge cases: BookKeeper's benchmark drops a zero where no value below zero can arise, and Kafka's tool replaces an arrival by the order of its two calls. Without them, 17 tools still cut. If the cuts made by a shipped script, or announced by the tool, are left out too, twelve do, eleven of them uncounted (Supplement S5). Of the five message tools that cut nothing, two time only to the acknowledgment, on a monotonic clock, and never meet such a value, and three keep it. We have not yet told the maintainers.

5.2 Tools run

Reading a tool shows what it does with a value below zero, not whether it ever meets one. We therefore ran 32 benchmarking tools, four rounds each, in blocks whose predictions were fixed from the tools' sources and made public before their runs (Supplement S5). Four tests followed. First, a delay was added to the path in steps. Of the tools the delay could reach, all but two reported a latency that rose 0.56–1.48 ms for each millisecond added, each time the

TABLE 3
What the message tools we read and ran do with a latency at or below zero. Grouped by what each tool times; only the first group can time a one-way latency across hosts. Rows run in the order of each group's heading, alphabetical within each outcome. Step: the smallest change in the latency a tool reports. Each tool was read at source (Section 5.1) and run in the registered campaign (Section 5.2), unless its row says otherwise. Kafka's two bundled tools count as one tool read but two run. †Done by a script or notebook the project ships, not by the tool. ‡Announced, with a line per value or a crash. A value counts as cut when a figure the tool reports leaves it out or replaces it, even in BookKeeper's benchmark, which cannot produce one below zero, and in Kafka's end-to-end tool, which never computes S. Stricter readings lower the count (Section 5.1). Source lines, versions and dates: Supplement S5.

Tool	Step	What happens to a value at or below zero
Publish to receipt, with timestamps that can come from two clocks: 13 tools; 5 drop, 1 refuses, 4 replace, 3 keep		
Apache Pulsar's performance client	1 ms	Dropped when below zero, uncounted; a zero is kept
librdkafka's `rdkafka_performance`	10 μs	Dropped‡ unless above zero, uncounted; no value in our runs reached zero, so their verdict tested nothing
OpenMessaging Benchmark	1 ms	Dropped by a filter unless above zero, uncounted, as is any message under 1 ms
RabbitMQ Stream PerfTest	1 ms	Dropped when below zero, uncounted, since June 2026; a zero is kept
Yahoo Streaming Benchmark	1 ms	Dropped† by the notebook analyzing its output file, uncounted; kept in that file
Apache ActiveMQ Artemis's `perf`	1 μs	Refused by its histogram library; the session rolls back and the run reads success; uncounted
`emqtt-bench`	1 ms	Replaced by zero in its printed average; kept in its histogram since 2025
Google's Pub/Sub load test	1 ms	Replaced by the fastest bin in its Java client, whose controller throws once half lie there; its Go, Python and Node clients panic, raise or lose it; uncounted
Kafka's Trogdor, end to end	1 ms	Replaced by its histogram's floor, uncounted; a negative throws, losing a producer's sample
Quiver	1 ms	Replaced by some 10^{19} ms when unsigned arithmetic wraps it, uncounted
`kgo-repeater`; RocketMQ benchmark	1 μs, 1 ms	Kept, unflagged; RocketMQ's maximum, which starts at zero, cannot show it
PerfKit Benchmarker, messaging	1 ms	Kept, floored to whole milliseconds, also when a step of the clock makes one
Publish to receipt, both timestamps in one process: 7 tools; 2 drop, 2 refuse, 3 replace		
Storage Benchmark Kit (SBK)	1 ms	Dropped from its figures but counted, a zero kept; the one message tool counting its cuts
Strimzi Canary	1 ms	Dropped† by the log reader it ships, uncounted; binned with the fastest messages
NATS command-line latency test	1 μs	Refused by its histogram, kept for minimum, median and maximum; uncounted
Pravega benchmark	1 ms	An array index that throws‡, ending every later statistic; uncounted
Kafka's end-to-end tool	1 ms	An arrival before its acknowledgment replaced by the time of the next poll, uncounted; under 1 ms every percentile reads zero
RabbitMQ PerfTest	1 μs	Replaced by its absolute value, uncounted; until June 2026 a zero was dropped too
Xinfra Monitor	1 ms	Replaced by zero in its percentiles, kept in its average; uncounted
Publish to the acknowledgment, in one process: 4 tools; 1 drops, 1 keeps, 2 never meet it		
Apache BookKeeper's benchmark	1 ns	A zero dropped with the failed writes, uncounted; no value below zero can arise
Kafka's producer tool	1 ms	Every value kept, unflagged
`mqtt-benchmark`; Sarama's tool	1 ns, 1 ms	Never met: a monotonic clock in one process; Sarama truncates every request under 1 ms to zero
Request to reply, a round trip in one process: 8 tools; 1 refuses, 7 never meet it		
`wrk2`	10 μs	Printed as impossible‡, then refused by its histogram, or the run aborts; uncounted
`hey`, `k6`, `memtier`, `nats bench`, Valkey benchmark, Vegeta, YCSB (all run, not read)	$\leq$0.1 ms	Cannot fall below zero in one process; none did when the clock was moved back

path crossed the delay, so those tools measure the path. Second, the clock of the process that subtracts was set back, by two offsets longer than the interval the tool times. The figures of the 19 one-clock tools stayed within a quarter of the offset in 151 of their 152 runs. All of them but PerfKit Benchmarker, whose two processes share one machine, take both timestamps in one process, where no clock offset can reach them. So does the Pravega benchmark, which the test could not place. These 19 one-process tools mostly time something other than a delivery: eight a round trip and four the acknowledgment alone. The rest time publish to receipt, but in one process, so they cannot see a separate receiving process's wait for a core. Another ten timestamp a message in one process and subtract in another, and in 73 of their 80 runs the moved clock shifted their figures or cut what they kept (Table 3). Third, the wall clock was stepped back under six tools that time on one clock, and each did with the value below zero what its source predicted, in all four rounds. Fourth, with every core 88% busy, the 99th percentile of each of the 25 tools whose figures can show a wait for a core rose by 0.9–7.0 ms, and real-time priority took nearly all of that rise back in 17 of them. In these tools a wait for a core shows as time added, not as a value below zero. The clock tests exposed the worst failures: the Pravega benchmark stops counting at the first value below zero yet exits as a success, the Pub/Sub load test's controller throws, blaming negative values, and Quiver's unsigned subtraction makes such a value some 10^{19} ms.

5.3 A positivity filter on a millisecond clock

The positivity filter is one of the cuts above, and the one that deletes more than the values below zero. In statistical terms it truncates the distribution at zero: what it removes is not counted, as it would be if censored. We measured it in depth in a broker benchmark whose end-to-end percentiles 43 public reports print, the OpenMessaging Benchmark [1]. Its filter was added in 2018 because its end-to-end latency "is calculated based on time difference of `System.currentTimeMillis()` on two different machines" and "can be negative" [18], and its histogram rejects

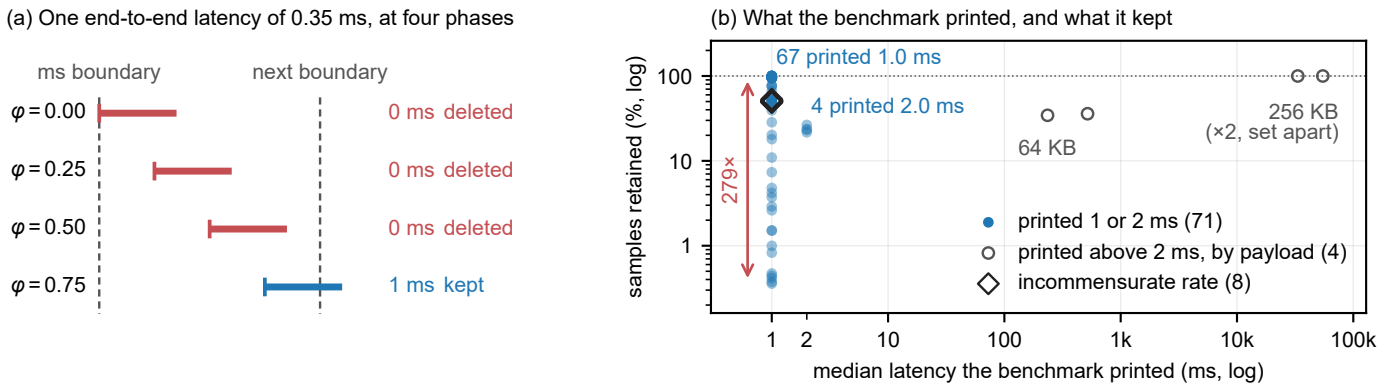


Fig. 4. **Samples that cross no millisecond boundary are deleted, and one printed median hides how much was kept.** (a) One end-to-end latency of 0.35 ms, starting at four phases φ within one millisecond: three compute to 0 ms and are deleted, and the fourth is kept, as exactly 1 ms. (b) In embedded mode, each of the 75 settings whose summary we captured, drawn at its printed median and *retention*, the share of samples kept, which spans $279\times$ among the 71 printing 1.0 or 2.0 ms. Diamonds: settings at two of the seven incommensurate rates of Section 5.3.

values below zero outright. For clocks that disagree, the filter is a sound local fix.

It has two side effects. Its clock counts whole milliseconds (whole seconds in its RabbitMQ driver; Supplement S5), so its *timestamp resolution* τ, the smallest step a timestamp can take, is a millisecond. A message that arrives before the millisecond digit changes therefore computes to zero and is deleted (Fig. 4a). And nothing counts what the filter deletes. A sample survives only if a millisecond boundary falls between its two timestamps. With publish instants placed uniformly within the millisecond, that happens with probability T_{true}/τ on a path shorter than τ, where T_{true}, the *true end-to-end latency*, is the one a perfect clock read without delay would give. So the share of samples kept, the *retention*, is

$$\text{retention} \;=\; \min\!\left(1,\; \frac{T_{\text{true}}}{\tau}\right). \tag{5}$$

Every sample kept on such a path reads exactly one step, so two brokers whose messages all arrive within a millisecond print the same median and differ only in their retention, which is never printed. The arithmetic is in Hewlett-Packard's 1970 counter note [19], but the deletion is not: that counter keeps its zero readings.

We added counters that record, by sign, every sample the filter discards, and changed nothing else. Every run used the benchmark's embedded mode, which runs it whole in one Java virtual machine. Of the 75 settings whose own summary we captured, 71 printed a median of exactly 1.0 or 2.0 ms, computed from 0.36–100% of the samples (Fig. 4b). Two replicates of one setting kept 431 and $120{,}434$ samples, 279-fold apart, and printed the same median. The filter can also inflate a median. At 64 KB, the two settings kept only 34.4–35.9% of samples, since most computed to zero, yet printed the slow remainder's medians, 235 and 519 ms.

Equation 5 holds where it can be tested. It predicts one retention for every configuration whose publish interval bears no ratio of small whole numbers to τ, an *incommensurate* rate. The seven such configurations all keep 47.2–51.5% at the median. That puts T_{true} at 0.49 ms, read as a mean over messages with any path longer than τ counted as τ. Where the ratio is one of small whole numbers, the schedule phase-locks with the clock, and retention takes few values; a registered change of message size moved a configuration onto one and off it (Supplement S4.3).

Counted by sign, the filter hides two failures. On Kafka the $10{,}913{,}263$ samples it discarded over 214 runs hold 0 values below zero; each of those samples computed to zero, lost to the clock's step. On Redis, where the benchmark subtracts the broker's millisecond timestamp from the consumer's clock, all $41{,}403$ values below zero in 9 runs were exactly $-\tau$, the mark of two floored clocks less than a millisecond apart (Supplement S4.4).

Public reports. An audit of the benchmark's public reports, registered before we read any, looked for the deletion's signature, a printed end-to-end median of exactly 1 ms. Of the 868 end-to-end medians in 43 reports, eight carry it, in two reports, and no report states how many samples it kept. The signature is rare partly because 9 reports come from copies with a finer clock (Supplement S5).

6 What to Do

The late reading, a clock read delayed while its thread waits for a core, and the deletion of values at or below zero are cheap to detect, and most of what they cost is cheap to repair (Table 4). One fix ends the late reading in a Linux tool that can read its sockets' timestamps, at the rates we tested: take both of a message's timestamps from the kernel, not from the tool's threads (Fig. 5). The other steps find or shrink it.

Let the kernel timestamp each message. A tool reads each of a message's timestamps in one of its threads, which may first wait for a core. The kernel timestamps each packet as it leaves and enters a machine, the transmit and receive timestamps, without waiting for any program's thread. A latency timed from a message's transmit timestamp to its receive timestamp, the *kernel-timed latency* W, therefore holds the network and the broker but no client thread's wait, whenever each socket read drains one packet and a consumer request already waits at the broker. No wait is lost: H_{out} runs from the publish call to the transmit timestamp and H_{in} runs from the receive timestamp to the consumer's reading, so $D = H_{\text{out}} + W + H_{\text{in}}$ for every message (Fig. 5). This is network measurement's rule: delay

TABLE 4
Most steps cost little, and most were measured. What a benchmark author or reader should do, and what each step buys on our runs.

Do this	What it buys
Time each message between the kernel's timestamps as it leaves and as it arrives, read on the tool's own sockets; across two machines, with clocks synchronized well below the latency.	Holds no client thread's wait at fifty messages a second: on one pair of machines, load raised the tail of D 4.2–4.7× and left this latency level, its median within 7.8% (Fig. 6; Supplement S3.10).
Sign-check every run under the load it is quoted at, and report the rejection rate.	Catches runs in which A exceeds D, and rejected 51.9% of our cloud runs. On a broker that delivers first, check the publish-timed spans alone.
Time from the publish call, or record the publish latency and add it back.	Recovers D exactly per message, and from medians alone to 0.4% on Kafka and 14.6% on Redis, where S alone understates it 3.8× and 6.6× (Supplement S2.1).
Give the threads that read the clock real-time priority, or have them spin on a dedicated core (*busy-polling*).	At least 4.4× fewer values of S below zero on Kafka and 4.2× on Redis, in every matched pair (Table 2). Busy-polling: not measured.
Record at native resolution, and print the retention and the discards counted by sign beside every latency.	Shows what a filter deleted, and whether the resolution or a clock failed (Section 5.3).
Read your own percentiles: on a fast path, a median of exactly one or two clock steps was set by the clock.	Flags 71 of the 75 settings we captured (Section 5.3).
Randomize the publish instants (Poisson sampling), and record the achieved rate.	Keeps the schedule from phase-locking with the clock; not measured directly, but dephased schedules keep Equation 5's retention (Section 5.3).

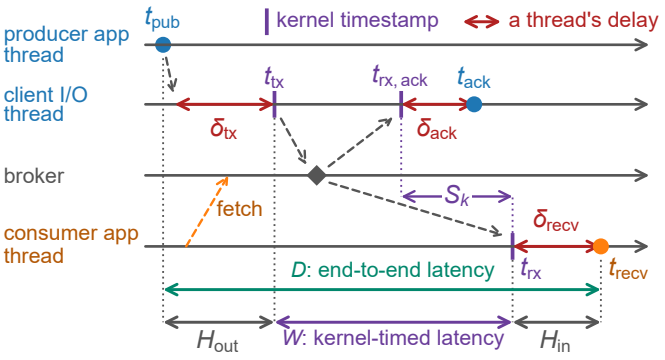


Fig. 5. **Timed between the kernel's timestamps, a message's latency holds no client thread's wait at fifty messages a second.** Kafka's path. The client's I/O thread writes the message after its delay δ_{tx}, and the kernel timestamps it as it leaves, t_{tx}; the consumer's request is already at the broker, which answers with the message. The acknowledgment and the message reach the client machine at $t_{rx,ack}$ and t_{rx}, before the delays δ_{ack} and δ_{recv}. $W = t_{rx} - t_{tx}$ holds the network and broker. As drawn, each client thread's wait on the message's path falls in H_{out} or H_{in}, and $S_k = t_{rx} - t_{rx,ack}$ holds no wait.

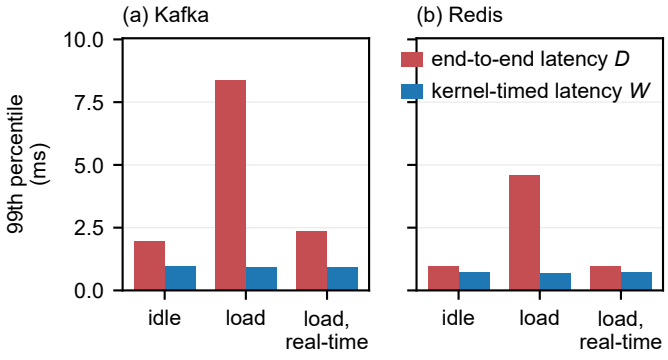


Fig. 6. **Timed by the kernel, a message's latency holds level under load, while the tail that benchmarks report grows 4.2–4.7-fold.** The 99th percentile of D, timed by the clients' threads, and of W, read on their own sockets, at fifty messages a second: on an idle client machine, under 88% background load at normal priority, and under that load at real-time priority, which takes most of D's rise back. Each bar is a setup's median over its runs (Supplement S3.10).

is defined between times on the wire, not the host times a program reads [20], and the one-way delay standard bounds each host's share by terms that H_{out} and H_{in} measure [7].

All five predictions, fixed before any run, held. Over 48 runs on both brokers, timed from packet captures, the median of W moved at most 7.9% and its 99th percentile 7.2% across load (none or 88%) and priority, inside the bands fixed before the runs, 10% and 25%. With six runs a setup, the run-level intervals on those moves reach 12% and 44%, so the bands hold for the point estimates, not their intervals (Supplement S3.9). In the same runs, load at normal priority raised the 99th percentile of D 4.3–4.6-fold. A delay added between broker and consumer raised W 0.93–0.95 ms per millisecond (Supplement S3.11), and W never reached zero, so a positivity filter deletes nothing.

Read those timestamps on the tool's own sockets. A tool need not capture packets. On Linux it reads these timestamps through the `SO_TIMESTAMPING` *socket option*, a setting a program makes on a socket, in four steps [21].

1) On each connection to the broker, before its first write, turn on the kernel's software timestamps for transmit and receive, with an identifier for each write counted in bytes (`OPT_ID` with `OPT_ID_TCP`).
2) The kernel leaves each write's transmit timestamp on the socket's *error queue*, keyed by the byte offset of the write's last byte; writes coalesced into one packet share one timestamp. Read the error queue at every write and read, since while it holds a timestamp a client that polls the socket spins.
3) Take the receive timestamp the kernel returns with each read, that of the last packet the read consumed.
4) Find each message in the bytes by an identifier the benchmark puts in each payload; this needs a connection without compression or encryption. Time it from the transmit timestamp of the write that carried it to the receive timestamp of the read that delivered it. Report that latency beside D, and count any message left without a timestamp.

Lancet reads a request-reply service's timestamps this

way [22], Fathom splits a fleet's call latency into host and network parts with kernel instrumentation [23], and Zuo et al. split a host's network-stack latency at kernel timestamps [24]. We read these timestamps beneath unmodified broker clients, on the one-way path from producer to consumer. Our Python module takes these steps for both brokers (`scripts/so_timestamps.py`). In 48 more runs it came within $3.0\,\mu$s of the captures' latency at the median. Both read the kernel's receive timestamp, so this shows the module matches timestamps to messages. The network card's own timestamps, 6–20 μs earlier at the median, bound how far the kernel's timestamps lie from the wire (Supplement S3.9). Under load at normal priority the tail of D again grew 4.2–4.7-fold while the latency the module timed held level, its median moving at most 7.8% across load and priority, with run-level intervals reaching 14% (Fig. 6). Against the capture runs, taking the timestamps moved the clients' own time, $D - W$, by -51 to $+76\,\mu$s on Kafka and $+15$ to $+45\,\mu$s on Redis, and the 99th percentile of D by -197 to $+183\,\mu$s, the drift between two days included (Supplement S3.10). Up to 2,000 messages a second the sockets agreed with the captures within $3.0\,\mu$s (Supplement S3.11).

What a tool must handle. A read that drains several packets times the earlier ones late, by their wait in the socket, which is the consumer's wait. A pull consumer, which asks the broker for each batch as Kafka's and Redis's do, adds another: a message the broker stores while no request is outstanding waits for the next one, which the consumer's thread issues after its own wait, inside W. A tool can flag such messages, those delivered in answer to a request whose transmit timestamp follows the message's own; Supplement S3.11 counts them at each rate. Where receive processing runs in a kernel thread, under threaded network polling or when an overloaded softirq moves to `ksoftirqd`, the receive timestamp waits for a core too. Reading a card's hardware timestamp avoids that wait, on a clock of its own that the tool must tie to the system's. Where a driver gives no software transmit timestamp, as on our cloud's accelerated network, ask for the one taken before the packet scheduler (`TX_SCHED`); any queueing there then falls inside W. Our virtual driver claims software timestamps it does not give (Supplement S3.10), so check which timestamps arrive. With producer and consumer on two machines, W spans two clocks and includes their offset, so they must be synchronized well below the latency. A design in which the consumer acknowledges back to the producer times two legs on one clock, not the one leg wanted. Recovering the two clocks' offset from logs instead is itself synchronization. Java's standard sockets expose neither the option nor the error queue, so a Java tool needs a native call, through the Java Native Interface or the foreign-function interface. On Linux 6.15 and later an eBPF program can replace the native call for transmit timestamps, not for receive timestamps. A kernel-bypass stack has no kernel timestamps at all. A tool that cannot yet take them needs what follows.

Sign-check every run. The *sign check* rejects a run when more than one percent of its messages carry a value below zero in any latency component it records (S, the publish delay or the processing-time latency), or when the median of any of them is below zero. It rejected 459 of the 884 cloud runs behind our results (51.9%; Supplement S2). It sees only the worst cases: the publish latency exceeds half the end-to-end latency on 77.6% of messages, but the check is triggered only by the 8.8% on which it exceeds the whole, and a latency that load inflates but leaves positive passes the check by construction. The check also assumes a broker that acknowledges no later than it delivers. On one that delivers first, as Redis often does, a value of S below zero needs no wait, so there it should read the publish-timed spans alone. The check therefore needs the repair that follows.

Time from the publish call, or add the publish latency back. The repair is Equation 1: $D = S + A$ for every message, and A is read on the producer's side of the same clock, so a benchmark that records its publish latency recovers each message's end-to-end latency exactly. Error arises only where a benchmark keeps summaries. Over groups of runs, the median of S plus the median of A misses the median of D by a median of 0.4% on Kafka's 32 groups and 14.6% on Redis's 38, and by 1.3 and 20% at the upper quartile (Supplement S2.1). On each broker, no difference is detectable between the groups of runs the sign check passes and those it rejects. The publish latency does not grow as leg 2 lengthens. On one path, the relative error of S as a latency, A/D, therefore falls as $1/D$. At our median publish latency, $A = 684\,\mu$s, it is 68% at an end-to-end latency of 1 ms and 6.8% at 10 ms, where the tenth to ninetieth percentiles of A give 5.1–19%. Below $D = 0.68$ ms, or 1.91 ms at the ninetieth percentile of A, the publish latency exceeds D itself. From one condition to the next, though, Kafka's A grows with D, so all its groups lie above that curve, while Redis's straddle it (Supplement S2.1 draws both). Over Kafka's 32 groups S understates D 3.8× at the median (interquartile range 3.8–4.0×), and over Redis's 38, 6.6× (4.8–7.2×), and over 24 matched workloads it inflates the gap between the brokers 1.7× at the median (Supplement S2.1). Timing from the publish call removes the values below zero, but not the consumer's own wait for a core, which lengthens every latency, unseen by any check. In 51 runs it added 0.41–0.94 ms to the mean of D, while the receiving thread's median wait was 18–46 μs (Supplement S3.8), a wait the kernel-timed latency excludes.

Give the threads that read the clock a core. Real-time priority cut the negative-span rate at least 4.4-fold on Kafka and 4.2-fold on Redis in every pair, at unchanged background load (Table 2). A busy-polling thread, spinning on a core of its own, never waits to be woken and should buy the same for the price of that core, though we did not measure it. Since Linux 6.12 a thread can instead ask for a shorter slice of its own, to run sooner when woken [25], which our Linux 6.8 kernels lack. At real-time priority the rate settles at 0.17–1.4% on Kafka and 1.8–2.5% on Redis (Supplement S3.1), so the sign check stays. A real-time or polling thread also changes the system it measures.

A better-synchronized clock repairs neither the late reading nor the deletion: the values of S below zero arise on one clock, and the deletion from the clock's step. A finer clock confines the deletion to faster paths but leaves the filter and every value of S below zero in place. The remedies are therefore ordered: record at native resolution, count what is discarded and randomize the publish instants, and only then improve the clocks' synchronization.

7 Related Work

Scheduling delay. That a ready thread can wait for a core is well known [14]. Behind another thread's slice the wait adds milliseconds to a server's tail latency, and real-time priority removes that wait [26]. Co-located work lengthens it [27], a longer slice extends it [28], and real-time Linux measures it as noise [29]. In Linux's network stack a reader's wait from its wakeup by arriving data to its running is the main source of tail latency [24], and the limits of the timers that benchmarks read are known too [30]. Load testers of microsecond-scale services correct for the *client bias* their own queueing and scheduling add [31], [22]. Villain et al. found that an outgoing socket timestamp "does not respect causality" under every stress pattern, including none [32]. None gives the consequence for a difference read by two processes: that it falls below zero at a rate set by the waiting probability and the margin, on the timescale of the slice.

Clocks. Across hosts, values below zero are read as clocks that disagree, and on one host as a thread preempted after it read the clock [33], but neither reading measures the rate or its laws. Sharma et al. read negative timing spans as clock skew, since "queueing alone cannot produce negative timing spans" [34]. For the backlog they model we agree, but a timestamping thread waits in a second queue, the kernel's run queue. Van Dongen and Van den Poel, seeing latencies below zero on up to a tenth of their events, took both timestamps from one broker [35], the one-clock rule reached by another route. Network measurement defines delay between wire times [7], and Lancet and Fathom read such times through the kernel [22], [23]. We read them beneath unmodified broker clients.

Resolution and discarded samples. The retention arithmetic and its cure, phase jitter, are in a 1970 counter note [19], and a clock coarser than the intervals it times can still estimate their mean [36]. IP measurement advises randomizing probe times [20], [37], since a periodic schedule can synchronize with periodic behavior [38], here the clock's step. Coordinated omission [39] is the deletion's mirror image: it fails to record samples that would have been slow. Network measurement reports the share of traces it rejects [40] and keeps a packet timestamped in the future [41], and rigorous benchmark reporting asks that removed outliers be counted [42], [43]. The tools we audited remove samples before any analyst sees them and count none, a case that neither that literature nor the principles for reproducible performance evaluation in the cloud [44] address. Stream-processing benchmarks compute latency from Kafka's millisecond timestamps of a record and of its result [45], so the retention arithmetic applies wherever their latencies fall below a millisecond. We add these checks, applied where a benchmark reports a latency.

8 Limitations

That scheduling makes Kafka's readings late is now observed message by message (Section 4.2). One clock excludes clock skew, and the manipulations exclude utilization and stress. Real-time priority excludes the interrupt noise that reaches every scheduling class, while timer slack, which differs between classes, cannot delay a wakeup on arriving data. Clocks that disagree across virtual CPUs are excluded too. On the Oracle machines a clock whose read costs at most 90 ns can only be one the kernel uses where the CPUs' clocks agree. The Azure machines read the processor's timestamp counter, which the kernel uses as its clock only where the counters agree (Supplement S3.6). Since D never fell below zero, no offset can exceed its smallest value, 455 times smaller than the deepest value of S. Both testbeds are virtual, so *steal time*, the time a virtual processor is ready but waits while the hypervisor runs another machine, is bounded, not excluded: real-time priority, a setting inside our virtual machine, cannot shorten it, yet it cut the rate. The rival we had left open, a broker that delivers before it acknowledges, is real on Redis and rare on Kafka (Section 4.1). By our registered rule, Redis's values of S below zero with the delivery first count as its reply order, though the readings' delays still decide which delivery-first messages fall below zero. That order lets Redis's real-time rate stay above zero: across the matched pairs it barely moved with load, 1.9% at the lower loads and 2.3% at the higher, while the normal-priority rate rose from 12% to 17% (Supplement S3.1). Our CPython timestamps also wait on the interpreter lock, and Kafka's Java client, which has none, shows values below zero too, at a third to a quarter of Python's rate (Supplement S6.2), so the effect needs no lock. With one publish worker instead of sixteen, Redis's real-time rate fell from 1.4% to 0.04% with no load, so there the lock the workers share sets most of it, though not under load (Supplement S3.11). We have not tested why placement moves the two brokers' rates in opposite directions, nor traced the cause of Redis's mode near $-200\,\mu$s in Fig. 2; on a second pair, a processor waking from idle did not cause it (Supplement S3.11).

Equation 3 describes how the rate moves between conditions. Inside one, the rate rises with D in 24 of 56 testable conditions, because a long delivery comes with a long publish latency (Supplement S3.4). A registered prediction that priority would cut the waiting probability at least tenfold failed: by the scheduler's counters, which average away the tail, it fell 1.7- to 2.2-fold, so the equation's factorization is untested between conditions. In the registered campaign, none of three predictions of the rate from recorded waits held within the registered factor of 1.5 on every pair and broker. Predictions from every thread's waits gave 2.49–13.08 times the measured rate. The share of readings below zero whose thread had mostly been waiting reached the registered 90% on four of twelve combinations. The law's fitted shape placed the fall in the slice's band less often than the free reading, and at no CPU count on Redis. Neither prediction of where the rate falls at a hand-set slice passed both readings, and those of how quickly it falls held on one broker at most (Supplement S6.2). Nor, with the kernel-timed margin, did the rate fall from each fifth of the margin to the next, as registered. Turning off the scheduler's run-to-parity rule, which lets a running thread keep its core until its slice is spent, moved neither the rate nor the acknowledging thread's delay, against a registered cut to two-thirds, so the rule alone does not hold a woken thread off the core (Supplement S3.11). The kernel trace pools both brokers' threads and is not free: under it Kafka's real-time runs read no value below zero, against 15 in $2{,}985$ untraced. The traced real-time configurations therefore cannot test the

trace check, though under the trace the normal-priority rate did not move (Supplement S3.5). The registered campaign's recording raised the rate it recorded 1.36- to 4.33-fold, so each prediction is held against its run's own rate. The base slice on the Oracle hosts is derived, not read off them; the Azure hosts read it back. The constants are those of one EEVDF-era kernel, before Linux 6.17 limited the run-to-parity rule, a change for threads that set their own slices. The distribution and the laws were measured with paced producers at about one message a second each, and fifty messages a second in the registered campaign, where the threads that read the clock sleep between messages and must be woken. At 500 messages a second on an idle host, two of eight runs had most values of S below zero for the whole run. The laws were not tested above fifty messages a second (Supplement S3.11). The rule that leaves out messages delayed by a testbed fault was chosen after the Oracle testbed was gone (Supplement S1.5).

The fix was tested from packet captures and, through a Python module, on the clients' own sockets. Those tests used one pair of machines and one kernel, with one partition, background load on the client machine alone, and the producer and the consumer on one machine. Its stillness under load was judged at fifty messages a second only. On a second pair, under load, Kafka's kernel-timed latency held within 6.3% and Redis's rose 26%. At $2{,}000$ messages a second, with 96.2–99.7% of messages flagged, a pull consumer's wait entered W. On that virtual network a departing packet's timestamp came only from before the packet scheduler, and the clients used neither encryption nor compression. We tested no designs that timestamp in hardware or pin a busy-polling thread to its own core, which keep the waiting state out of the latency they time.

The deletion is demonstrated in one benchmark, in its embedded mode, and reproduced in an independent program of ours; the benchmark's distributed mode, run on the Azure machines, dropped values below zero under a moved clock, uncounted (Supplement S4.5). In the registered change of message size (Supplement S4.3), the 32 KB configuration pinned at 69.2% retention, not the predicted 66.67%. The audit found the deletion's signature in few public results, so we claim it for those and our own runs only. We chose the tools in rounds, not by sampling, classified the 27 tools by reading them, and ran the 32 tools on a single kernel and machine pair, at one publish rate. How much a deployment loses depends on its end-to-end latency and publish rate, so we recommend reporting retention rather than calling any reported number wrong.

9 Conclusion

A latency benchmark measures the system only while the delay before each reading is short beside the latency it measures. On one machine's clock, the moment a producer reads a broker's acknowledgment and the moment a consumer reads the message can differ by milliseconds, in either order. A thread waits for a core, for up to a scheduler's slice, before it reads the clock, and a broker that delivers first needs no wait. Across the industry such values are deleted, replaced or never measured, a positivity filter on a millisecond clock deletes the fastest messages too, and no report we audited says how many samples it kept. On Linux a tool can time each message between the kernel's timestamps, read on its own sockets, and report that latency beside the end-to-end latency. At fifty messages a second the kernel-timed latency held no client thread's wait. Before acting on a broker latency below a few milliseconds, ask what it was timed from, how many samples the tool kept, and how many it discarded, by sign.

Acknowledgment

The authors thank N. Herbst, for the load-generation point that became one of the checks above, for pointing us to the timer study cited in Section 7, and for a reading that reset the abstract.

In accordance with IEEE policy, Anthropic's Claude assisted with code development and verification, literature search, and the drafting and editing of text in every section, under the authors' direction and review. Experimental design, scientific claims, and interpretation are the authors'.

Artifact Availability

Code, analysis and manuscript are archived at 10.5281/zenodo.21650031, the measurement dataset at 10.5281/zenodo.21650064; both resolve to the current version, `v4.0.0`, whose files are restricted pending publication. Every number here is recomputed from the committed data by the archived code at build time, and the test suite fails if the text and the data disagree.

References

[1] OpenMessaging Project, Linux Foundation, "The OpenMessaging benchmark framework," 2018, available at openmessaging.cloud/docs/benchmarks/. Accessed: Aug. 19, 2026.

[2] Redis, "Redis streams: Latency tests results," https://redis.io/docs/latest/develop/data-types/streams/, 2026, accessed: Oct. 15, 2026.

[3] L. Barroso, M. Marty, D. Patterson, and P. Ranganathan, "Attack of the killer microseconds," *Commun. ACM*, vol. 60, no. 4, pp. 48–54, Apr. 2017.

[4] J. Dean and L. A. Barroso, "The tail at scale," *Commun. ACM*, vol. 56, no. 2, pp. 74–80, Feb. 2013.

[5] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!" in *Proc. 14th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Mar. 2009, pp. 265–276.

[6] M. Baker, "1,500 scientists lift the lid on reproducibility," *Nature*, vol. 533, no. 7604, pp. 452–454, May 2016.

[7] G. Almes, S. Kalidindi, M. Zekauskas, and A. Morton, Ed., "A one-way delay metric for IP performance metrics (IPPM)," Internet Engineering Task Force, RFC 7679, Jan. 2016, STD 81.

[8] J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen, and V. Markl, "Benchmarking distributed stream data processing systems," in *Proc. IEEE 34th Int. Conf. Data Engineering (ICDE)*, Apr. 2018, pp. 1507–1518.

[9] I. Stoica, H. Abdel-Wahab, K. Jeffay, S. K. Baruah, J. E. Gehrke, and C. G. Plaxton, "A proportional share resource allocation algorithm for real-time, time-shared systems," in *Proc. 17th IEEE Real-Time Systems Symp. (RTSS)*, Dec. 1996, pp. 288–299.

[10] StatsBomb, "Statsbomb open data," *GitHub Repository*, 2023, accessed: Oct. 15, 2026. [Online]. Available: https://github.com/statsbomb/open-data

[11] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor, "The preregistration revolution," *Proc. Natl. Acad. Sci. USA*, vol. 115, no. 11, pp. 2600–2606, Mar. 2018.

[12] OpenMessaging Benchmark contributors, "Can the average publish latency be larger than end-to-end latency? (issue #216)," https://github.com/openmessaging/benchmark/issues/216, 2021, accessed: Oct. 15, 2026.

[13] D. Bristot de Oliveira, D. Casini, J. Lelli, and T. Cucinotta, "Timerlat: Real-time Linux scheduling latency measurements, tracing, and analysis," *IEEE Trans. Comput.*, vol. 74, no. 8, pp. 2608–2620, Aug. 2025.

[14] B. Gregg, "Linux bcc/BPF run queue (scheduler) latency," https://www.brendangregg.com/blog/2016-10-08/linux-bcc-runqlat.html, Oct. 2016, accessed: Sep. 13, 2026.

[15] Z. Zhou, "sched: Reduce the default slice to avoid tasks getting an extra tick," Linux kernel source, commit 2ae891b82695, first released in v6.15, 2025, https://github.com/torvalds/linux/commit/2ae891b826958b60919ea21c727f77bcd6ffcc2c. Accessed: Oct. 15, 2026.

[16] Broadcom, "RabbitMQ PerfTest — `TimestampProvider.java` and `DefaultPerformanceMetrics.java`," Source repository, 2026, https://github.com/rabbitmq/rabbitmq-perf-test. Accessed: Oct. 2, 2026.

[17] Apache Software Foundation, "Apache Kafka tools — `EndToEndLatency.java` and `ProducerPerformance.java`," Source repository, 2026, https://github.com/apache/kafka. Accessed: Aug. 31, 2026.

[18] S. Guo, "Avoid adding negative metric values (pull request #56)," OpenMessaging Benchmark, commit ebb5d6b8, Mar. 2018, https://github.com/openmessaging/benchmark/pull/56. Accessed: Aug. 18, 2026.

[19] Hewlett-Packard, "Time interval averaging," Hewlett-Packard Company, Application Note 162-1, 1970, part no. 02-5952-0766.

[20] V. Paxson, G. Almes, J. Mahdavi, and M. Mathis, "Framework for IP performance metrics," Internet Engineering Task Force, RFC 2330, May 1998.

[21] Linux Kernel Documentation, "Timestamping," https://docs.kernel.org/networking/timestamping.html, 2026, accessed: Oct. 15, 2026.

[22] M. Kogias, S. Mallon, and E. Bugnion, "Lancet: A self-correcting latency measuring tool," in *Proc. USENIX Annual Technical Conf. (ATC)*, Jul. 2019, pp. 881–896.

[23] M. A. Qureshi, J. Yan, Y. Cheng, S. Hassas Yeganeh, Y. Seung, N. Cardwell, W. De Bruijn, V. Jacobson, J. Kaur, D. Wetherall, and A. Vahdat, "Fathom: Understanding datacenter application network performance," in *Proc. ACM SIGCOMM 2023 Conf.*, Sep. 2023, pp. 394–405.

[24] T. Zuo, J. Hwang, A. Tang, R. Agarwal, and Q. Cai, "Understanding host network stack latency," in *Proc. ACM SIGCOMM 2026 Conf.*, Aug. 2026, pp. 1127–1140.

[25] P. Zijlstra, "sched/eevdf: Use sched_attr::sched_runtime to set request/slice suggestion," Linux kernel source, commit 857b158dc5e8, first released in v6.12, 2024, https://github.com/torvalds/linux/commit/857b158dc5e81c6de795ef6be006eed146098fc6. Accessed: Oct. 15, 2026.

[26] J. Li, N. K. Sharma, D. R. K. Ports, and S. D. Gribble, "Tales of the tail: Hardware, OS, and application-level sources of tail latency," in *Proc. ACM SoCC*. ACM, Nov. 2014, pp. 1–14.

[27] J. Leverich and C. Kozyrakis, "Reconciling high server utilization and sub-millisecond quality-of-service," in *Proc. 9th European Conf. Computer Systems (EuroSys)*, Apr. 2014, pp. 1–14.

[28] Y. Jia, K. Tian, Y. You, Y. Chen, and K. Chen, "Skyloft: A general high-efficient scheduling framework in user space," in *Proc. 30th ACM Symp. Operating Systems Principles (SOSP)*, Nov. 2024, pp. 265–279.

[29] D. Bristot de Oliveira, D. Casini, and T. Cucinotta, "Operating system noise in the Linux kernel," *IEEE Trans. Comput.*, vol. 72, no. 1, pp. 196–207, Jan. 2023.

[30] M. Kuperberg, M. Krogmann, and R. H. Reussner, "Metric-based selection of timer methods for accurate measurements," in *Proc. 2nd ACM/SPEC Int. Conf. Performance Engineering (ICPE)*, Mar. 2011, pp. 151–156.

[31] Y. Zhang, D. Meisner, J. Mars, and L. Tang, "Treadmill: Attributing the source of tail latency through precise load testing and statistical inference," in *Proc. 43rd ACM/IEEE Int. Symp. Computer Architecture (ISCA)*, Jun. 2016, pp. 456–468.

[32] B. Villain, M. Davis, J. Ridoux, D. Veitch, and N. Normand, "Probing the latencies of software timestamping," in *Proc. IEEE Int. Symp. Precision Clock Synchronization for Measurement, Control and Communication (ISPCS)*, Sep. 2012, pp. 1–6.

[33] P. E. McKenney, "Is parallel programming hard, and, if so, what can you do about it?" arXiv preprint arXiv:1701.00854v6, Jun. 2023, release v2023.06.11a, App. A.3–A.4.

[34] A. Sharma, D. Shah, D. Lariviere, and H. ElBakoury, "Time, causality, and observability failures in distributed AI inference systems," *arXiv preprint arXiv:2604.21361*, Apr. 2026, Open Compute Project, Unified Intelligent Infrastructure workstream.

[35] G. Van Dongen and D. Van den Poel, "Evaluation of stream processing frameworks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 8, pp. 1845–1858, Aug. 2020.

[36] P. B. Danzig and S. Melvin, "High resolution timing with low resolution clocks and a microsecond resolution timer for sun workstations," *ACM SIGOPS Operating Systems Review*, vol. 24, no. 1, pp. 23–26, Jan. 1990.

[37] F. Baccelli, S. Machiraju, D. Veitch, and J. C. Bolot, "The role of PASTA in network measurement," in *Proc. ACM SIGCOMM*, Sep. 2006, pp. 231–242.

[38] V. Raisanen, G. Grotefeld, and A. Morton, "Network performance measurement with periodic streams," Internet Engineering Task Force, RFC 3432, Nov. 2002.

[39] G. Tene, "How NOT to measure latency," 2015, talk, Strange Loop 2015, St. Louis, MO, USA. https://www.youtube.com/watch?v=lJ8ydIuPFeU.

[40] V. Paxson, "On calibrating measurements of packet transit times," in *Proc. ACM SIGMETRICS*. ACM, Jun. 1998, pp. 11–21.

[41] S. Shalunov, B. Teitelbaum, A. Karp, J. W. Boote, and M. J. Zekauskas, "A one-way active measurement protocol (OWAMP)," Internet Engineering Task Force, RFC 4656, Sep. 2006.

[42] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming, Systems, Languages and Applications (OOPSLA)*. ACM, Oct. 2007, pp. 57–76.

[43] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*. ACM, Nov. 2015, pp. 73:1–73:12.

[44] A. V. Papadopoulos, L. Versluis, A. Bauer, N. Herbst, J. von Kistowski, A. Ali-Eldin, C. L. Abad, J. N. Amaral, P. Tůma, and A. Iosup, "Methodological principles for reproducible performance evaluation in cloud computing," *IEEE Trans. Softw. Eng.*, vol. 47, no. 8, pp. 1528–1543, Aug. 2021.

[45] S. Henning, A. Vogel, M. Leichtfried, O. Ertl, and R. Rabiser, "ShuffleBench: A benchmark for large-scale data shuffling operations with distributed stream processing frameworks," in *Proc. 15th ACM/SPEC Int. Conf. Performance Engineering (ICPE)*, May 2024, pp. 2–13.

Gustavo Pedro Ricou received the B.Sc. degree in physics from the Universidade do Porto, Portugal, and specialized in nuclear fusion science at the Universität Stuttgart, Germany, and the Université de Lorraine, France. He holds an M.Sc. in software design with cloud-native computing from the Technological University of the Shannon, with first-class honors, and is currently pursuing an M.Sc. in statistics and data science at Trinity College Dublin, Ireland. He is a Software Engineer with BoscaSports. His research concerns measurement validity — what a reported number is really evidence of — in computer systems, environmental measurement and sports analytics, using pre-registration and physical-consistency auditing.

Romaric Duvignau received the Ph.D. degree in computer science from the University of Bordeaux, France, in 2015, for work on the maintenance and simulation of dynamic random graphs. He is an Associate Professor and Head of the Network and Systems unit at Chalmers University of Technology, Gothenburg, Sweden. His research concerns distributed and data-driven algorithms, the monitoring of large-scale distributed systems, and stream analytics for cyber-physical systems.